```
import random
import time
import numpy as np
from tabulate import tabulate
```

---------- PARTITION ----------

```
def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1
```

---------- RANDOMIZED SELECT ----------

```
def randomized_select(arr, k):
    arr = arr.copy()
    low, high = 0, len(arr) - 1
    while low < high:
        pivot_index = random.randint(low, high)
        arr[pivot_index], arr[high] = arr[high], arr[pivot_index]
        pivot_pos = partition(arr, low, high)
        rank = pivot_pos - low + 1
        if k == rank:
            return arr[pivot_pos]
        elif k < rank:
            high = pivot_pos - 1
        else:
            low = pivot_pos + 1
            k -= rank
    return arr[low]
```

---------- DETERMINISTIC SELECT ----------

```
def get_mom(arr):
    if len(arr) <= 5:
        return sorted(arr)[len(arr)//2]
    medians = [get_mom(arr[i:i+5]) for i in range(0, len(arr), 5)]
    return get_mom(medians)

def deterministic_select(arr, k):
    arr = arr.copy()
    low, high = 0, len(arr) - 1
    while low < high:
        mom = get_mom(arr[low:high+1])
        # Swap mom to end for partition
        for i in range(high, low-1, -1):
            if arr[i] == mom:
                arr[i], arr[high] = arr[high], arr[i]
                break
        pivot_pos = partition(arr, low, high)
        rank = pivot_pos - low + 1
        if k == rank:
            return arr[pivot_pos]
        elif k < rank:
            high = pivot_pos - 1
        else:
            low = pivot_pos + 1
            k -= rank
    return arr[low]

#testing
def verify_correctness():
    test_cases = [
        # ([3, 1, 4, 1, 5, 9, 2, 6], 4, 4),
        # ([10, 9, 8, 7, 6, 5, 4, 3, 2, 1], 5, 5),
        # ([1, 2, 3, 4, 5], 3, 3),
        # ([5, 4, 3, 2, 1], 1, 1),
        # ([100, 200, 300, 400, 500], 5, 500),
        # ([1,1,1,1,1], 3, 1),  # All duplicates
    ]
    for arr, k, expected in test_cases:
        det = deterministic_select(arr, k)
        rand = randomized_select(arr, k)
        status = "PASS" if det == expected and rand == expected else
"FAIL"
        print(f"{status}: k={k}, Expected={expected}, Det={det},
Rand={rand}")
```

```python
def test_performance(sizes=[100,500,1000,3000,5000], num_tests=5):
    results = []
    for size in sizes:
        det_times, rand_times = [], []
        for _ in range(num_tests):
            arr = [random.randint(1, 1000000) for _ in range(size)]
            k = random.randint(1, size)
            # Det
            start = time.perf_counter()
            deterministic_select(arr, k)
            det_times.append(time.perf_counter() - start)
            # Rand
            start = time.perf_counter()
            randomized_select(arr, k)
            rand_times.append(time.perf_counter() - start)
        results.append({
            'size': size,
            'det_avg': np.mean(det_times),
            'det_std': np.std(det_times),
            'rand_avg': np.mean(rand_times),
            'rand_std': np.std(rand_times),
            'speedup': np.mean(det_times) / np.mean(rand_times)
        })
    # Print table
    headers = ['Size', 'Deterministic (s)', 'Std Dev', 'Randomized
(s)', 'Std Dev', 'Speedup']
    table = [[r['size'], f"{r['det_avg']:.6f}", f"{r['det_std']:.6f}",
              f"{r['rand_avg']:.6f}", f"{r['rand_std']:.6f}",
f"{r['speedup']:.2f}x"]
             for r in results]
    print("\n" + "="*80)
    print("PERFORMANCE COMPARISON")
    print("="*80)
    print(tabulate(table, headers=headers, tablefmt='grid'))

verify_correctness()
test_performance()
```

```
==========================================================================
==========
PERFORMANCE COMPARISON
==========================================================================
==========
+--------+---------------------+-----------+-----------------
+-----------+-----------+
|   Size |   Deterministic (s) |   Std Dev |   Randomized (s) |   Std
Dev | Speedup   |
+========+=====================+===========+==================+=======
====+===========+
```

```
|    100 |            4e-05    |  1.2e-05  |         1.6e-05  |  4e-06
| 2.52x     |
+--------+---------------------+-----------+------------------
+-----------+-----------+
|    500 |            0.00022  |  8.4e-05  |         7.6e-05  |  2.8e-
05  | 2.91x     |
+--------+---------------------+-----------+------------------
+-----------+-----------+
|   1000 |            0.000367 |  0.000111 |         0.00013  |  5.2e-
05  | 2.81x     |
+--------+---------------------+-----------+------------------
+-----------+-----------+
|   3000 |            0.001025 |  0.00011  |         0.000607 |
0.00027  | 1.69x     |
+--------+---------------------+-----------+------------------
+-----------+-----------+
|   5000 |            0.001908 |  0.000432 |         0.001421 |
0.001274 | 1.34x     |
+--------+---------------------+-----------+------------------
+-----------+-----------+




# -------------------- TESTING --------------------



==========================================================================
==========
PERFORMANCE COMPARISON
==========================================================================
==========
+--------+---------------------+-----------+------------------
+-----------+-----------+
|   Size |   Deterministic (s) |   Std Dev |   Randomized (s) |   Std
Dev | Speedup   |
+========+=====================+===========+==================+=======
====+===========+
|    100 |            4.1e-05  |  7e-06    |         2.1e-05  |  5e-06
| 1.93x     |
+--------+---------------------+-----------+------------------
+-----------+-----------+
|    500 |            0.000164 |  1e-05    |         7.7e-05  |  1e-05
```

```
| 2.12x     |
+--------+---------------------+-----------+------------------
+-----------+-----------+
|   1000 |            0.000335 |  4.4e-05  |         0.000112 |  2.2e-
05  | 2.99x     |
+--------+---------------------+-----------+------------------
+-----------+-----------+
|   3000 |            0.000935 |  6.1e-05  |         0.000526 |
0.000127 | 1.78x     |
+--------+---------------------+-----------+------------------
+-----------+-----------+
|   5000 |            0.001735 |  0.000291 |         0.001015 |
0.00058  | 1.71x     |
+--------+---------------------+-----------+------------------
+-----------+-----------+
```